

WHITEPAPER • V2.0

Compute You Can Prove

A control plane for agent compute: an OpenAI- and Anthropic-compatible API plus metered code sandboxes, where every unit of work is metered at the gateway, bounded by a budget the caller sets, paid in prepaid stablecoin, and signed on the way out as a receipt anyone can verify.

VERSION

2.0 — August 2026

SUPERSEDES

v1.0 — May 2026

CATEGORY

Agent compute infrastructure

WEB

buildsable.com

ABSTRACT

Autonomous agents consume compute the way processes consume CPU: continuously, unattended, and without a human in the loop to notice when something goes wrong. Two failures follow. The first is disclosure — an agent’s prompts and the code it executes are its entire working memory, streamed to counterparties who log it. The second is unaccountability — nobody can say afterwards what actually ran, on what, for how much. **Sable Network** is the control plane that closes both: a drop-in OpenAI- and Anthropic-compatible endpoint plus metered code-execution sandboxes, where inbound payloads are sealed on arrival, nothing content-bearing is ever persisted, every unit of work is metered and pre-authorized against a hard budget, settlement is prepaid stablecoin or inline x402, and every response carries a secp256k1-signed receipt that a third party can verify without trusting us. On the confidential tier, that receipt additionally carries a hardware attestation the buyer can check before sending anything, and the response is cryptographically bound to the enclave that produced it. This paper describes what is built, what is not, and the direction: the compute underneath the surface becomes machines Sable does not own — which is possible only because the trust does not live in the machine.

CONTENTS

01	The delegation problem	08	Confidential execution & attestation
02	The control plane	09	Sandbox compute
03	The privacy contract	10	What is built, and what is not
04	Architecture	11	Direction: compute for rent
05	Metering, budgets & delegation	12	Threat model & trust boundaries
06	Payment: prepaid USDT and x402	13	Conclusion
07	Verifiable receipts	A	Appendix: API surface

01 The delegation problem

You cannot delegate authority to a system you can neither keep private nor hold to account.

When a person uses a model, they disclose something: a question, a draft, a fragment of a plan. When an *autonomous agent* uses one, it discloses everything — its objectives, its tools, its private state, the contents of the documents it is reasoning over, and increasingly the code it wants executed and the secrets that code needs. It does this not once per session but continuously, as a condition of functioning.

Disclosure is only the first half of the problem, and it is the half the industry has started to name. The second half is quieter and, operationally, bites sooner. An agent left running is a process with a wallet. It can loop. It can spawn sub-agents. It can be prompt-injected into doing something expensive. And when the bill arrives, the questions that matter — what ran, on what hardware, under whose budget, and can you prove it — have no answer, because the systems in the middle were built for a human clicking a button, not for a machine spending money unsupervised.

Two properties make delegation safe: the delegate cannot leak what it was given, and the principal can bound and later verify what it did. Most infrastructure offers neither. Both are engineering problems, not policy problems.

Sable Network addresses both at the same layer, because they are the same layer. The wedge is deliberately narrow: a developer adopts it by changing one line.

```
# Before
client = OpenAI()

# After - sealed on ingress, metered, budget-bounded, receipted
client = Sable(base_url="https://api.buildsable.com/v1")
```

Everything else — chat completions, embeddings, streaming, tool use — keeps working unchanged, and the Anthropic Messages API is served from the same gateway, so an Anthropic-shaped client works by pointing `ANTHROPIC_BASE_URL` at it. Wire compatibility is not merely convenience; it is **portability insurance**. If Sable disappeared tomorrow, the same code runs against any other compatible endpoint with one URL change. Trust that requires lock-in is not trust — it is capture.

02 The control plane

Sable does not sell you a machine. It sells the metering, the budget, and the proof that sit over one.

The product is a gateway, and the gateway is the whole thesis. It accepts two kinds of work through one credential, one balance, and one policy surface:

- **Inference** — chat completions and embeddings, metered in tokens.
- **Sandboxes** — short-lived code execution, metered in vCPU-seconds and gigabyte-seconds.

Both are billed against the same prepaid balance, bounded by the same per-key spend caps, subject to the same resource allowlists and expiry, recorded in the same metadata-only event table, and signed with the same receipt key. An agent that can run inference and execute code needs one credential and one budget, not two vendors and two invoices.

The catalog is published under **stable Sable IDs**, decoupled from upstream naming churn. A few of them are load-bearing: the flagship family — `sable`, `sable-fast`, `sable-max` — where each id is kept pointed at a strong frontier engine (currently Claude Opus 5 behind `sable`), disclosed rather than hidden: each flagship id carries a chain of engines it can fail over across, and the engine that actually served is recorded in every signed receipt. Around it sits a slate of frontier models served through the gateway, where the upstream provider sees Sable as the caller rather than the buyer. That anonymization is worth having and worth being precise about: **anonymized access is not the confidential tier**. Only TEE-attested models are confidential; on every other model the upstream host sees the plaintext, exactly as the privacy contract states. Between the two sits a **double-blind private lane**: ids pinned to a second, privacy-focused intermediary, so the request reaches the model through a provider that sees Sable rather than the caller. A pinned route never fails over — a failover that changed the privacy path would be worse than an error — and the intermediary's default system prompt is disabled at egress.

What makes this a *control plane* rather than a reseller is where authority sits. Metering is computed at the gateway from what the gateway observed, not from what a backend reported it did. Budgets are enforced before an upstream call is made, not reconciled afterwards. Attestation is verified against a pinned measurement by the gateway, and if verification fails the request is refused rather than quietly downgraded. The backend is, deliberately, the least trusted component in the system — which is precisely what makes it replaceable.

THE DESIGN CONSTRAINT

Every feature must compose with the OpenAI-compatible surface rather than replace it. A caller written against the gateway on its first day keeps working as the compute underneath changes — different upstream providers, attested enclaves, and eventually machines Sable does not own. The interface is the contract; the substrate is an implementation detail. This is what lets the supply side change without breaking the demand side.

03 The privacy contract

A short list of invariants, enforced in code and testable from outside.

Sable's guarantees are expressed as a small set of blunt invariants that the gateway is built around, rather than as a policy document that describes intentions.

- **Prompts, completions, and submitted code are never written to disk or logs.** Plaintext exists only inside named, in-frame code paths: the egress shim immediately before an upstream model call, the request-scoped sandbox execution path, and the translation frame that adapts Anthropic-shaped requests onto the chat handler. Adding a new plaintext surface requires amending that inventory.
- **Inbound payloads are sealed on ingress.** The request body is serialized and encrypted with AES-256-GCM before any routing decision is taken and before any network egress.
- **Persistence is metadata only.** One event row per unit of work: kind, unit, quantity, resource, model, provider, tier, node, token counts, cost, latency, status, error class, timestamp. The sandbox table has *no column* that could hold code, stdout, stderr, or produced files. The only content-derived value stored anywhere is a truncated SHA-256 fingerprint.
- **Credentials are stored as Argon2id hashes.** The plaintext of an API key is returned exactly once at creation and is never re-derivable; a short prefix is kept in the clear only for labelling.
- **Telemetry respects the contract.** Request IDs, key IDs, model strings, token counts, latencies, and content *fingerprints* are loggable. Prompt text, message arrays, completion content, and upstream response bodies are not. Upstream 4xx bodies are reduced to content-free error classes at the provider boundary, because a moderation or validation rejection can echo the prompt back at you.
- **Secrets live in the environment.** The master key is required at boot; the service refuses to start without one.
- **The portal never displays content.** History shows metadata. It cannot show a prompt, because the prompt was never kept.

The discipline that makes these real is a single question asked of every change: *could a malicious operator, or a curious engineer with production access, reconstruct the payload from what I am about to add?* If yes, it does not ship. Two consequences are worth stating plainly, because they are costs and not features. Abuse response is necessarily **account-level rather than content-level** — we cannot search content we never stored, so reports are actioned against accounts and against fingerprints a reporter supplies, never by scanning traffic. And a sandbox run that is retried with an idempotency key **replays its receipt, not its output**: the metadata was recorded, the output never was.

04 Architecture

Seal on ingress, authorize against a budget, dispatch once, meter what happened, sign the result.

The control plane is a single deployed component: an HTTP gateway speaking the OpenAI wire protocol, with an Anthropic-shaped façade and an MCP server on the same port. Sandbox work is dispatched from it to an enrolled runner node — the gateway claims a per-job concurrency lease transactionally before it dials anything. Its internal flow is the product.

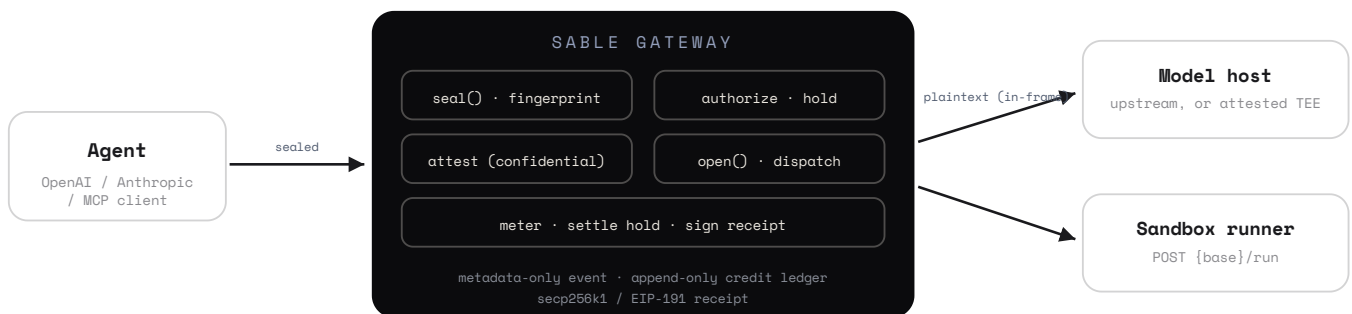


FIG. 1 — ONE PIPELINE, TWO KINDS OF WORK. PLAINTEXT EXISTS ONLY BETWEEN OPEN() AND THE DISPATCH IT WAS OPENED FOR.

Envelope encryption

On ingress the request body is serialized and sealed with **AES-256-GCM** under a fresh 96-bit nonce. The sealed envelope, not the plaintext, is what the routing layer carries. The same primitive gives integrity: any tampering with the ciphertext fails authentication on open. A short SHA-256 fingerprint is computed pre-seal for safe logging and correlation — it identifies a request without revealing it, and it is the value that later appears on the receipt.

The egress shim

Decryption happens exactly once, immediately before the dispatch it exists for, inside a deliberately small code path. This is the entire privacy surface. The plaintext does not leave that frame: it is not logged, not persisted, not echoed into errors, and not retained once the response has streamed back. Backend error text is reduced to fixed classes rather than forwarded, because a backend's own error message is one of the easiest ways to leak a payload — or, as we learned by shipping it, the gateway host's filesystem layout. An opt-in scrub (`sable_scrub`) can additionally redact secret- and PII-shaped spans — emails, wallet addresses, API-key shapes, long hex — from the outbound payload inside this same frame, before it reaches any vendor; nothing scrubbed or unscrubbed is persisted, and only the redaction count is ever traced.

Metadata-only accounting

What Sable retains per unit of work is one row:

```
usage_events (  
  kind · unit · quantity · resource  
  model · provider · privacy_tier · node_id  
  prompt_tokens · completion_tokens · total_tokens · cost_micro_usd  
  latency_ms · status · error_class · created_at  
)  
// absent by construction: prompts, completions, embeddings, submitted  
// code, stdout, stderr, or anything they could be reconstructed from.
```

One table describes tokens and vCPU-seconds alike — the `kind / unit / quantity` triple is what makes a single billing, budgeting, and reporting path work across both. The portal, the usage stream, spend caps, and the invoice all read this row. None of them can surface content, because content was never stored.

05 Metering, budgets & delegation

A budget that is checked after the money is spent is not a budget.

Credit is an **append-only ledger**. A balance is the sum of its rows; nothing is updated in place, so the audit trail cannot be rewritten by the code that spends against it. A cached balance column is maintained inside the same database transaction as every ledger append, purely as an index — a reconciliation job recomputes the sum on a schedule and *alerts on drift without silently repairing it*, because a self-healing cache destroys the evidence of the bug that caused the drift.

Spending is **pre-authorized, not post-paid**. Before any upstream call the gateway places a *hold* for the request's worst-case cost — for a sandbox, every second of its declared timeout at its declared vCPU and memory; for inference, an estimate floored at a configured minimum. Spendable credit is the balance minus outstanding holds. The hold is released at settlement, on every error path, and by a sweeper if the process dies while holding one. This closes a race that a simple balance check does not: without holds, a balance of one micro-dollar authorizes an unbounded number of concurrent jobs, which is exactly the failure mode a runaway agent produces.

Debits are **idempotent at the database**, enforced by a partial unique index on the usage reference rather than by application memory, so a retried metering write cannot double-bill. Side-effecting calls get a second layer: `POST /v1/sandboxes` accepts an `Idempotency-Key` that is claimed by a row insert *before* execution, so a client retry cannot execute the code twice.

Per-key controls

Agents fail in ways human users do not, so every key carries optional, enforced limits: a monthly **spend cap** (over it, 402 before any upstream call), a **model and resource allowlist** (outside it, 403), an **expiry** (past it, 401), and a per-key **rate limit** (over it, 429 with `Retry-After`).

Delegated sub-keys

The interesting case is an agent that spawns sub-agents mid-run, with no human available to mint credentials. `POST /v1/keys/delegate` is authenticated by the *parent API key* rather than a wallet session, and issues a child that is **≤ its parent on every axis**. Two rules are deliberately the opposite of the obvious implementation, because the obvious implementation silently defeats the feature:

- **Omitting a field inherits the parent's limit** — it does not mean "unlimited".
- **An allowlist intersects**. A child requesting models its parent cannot use is refused, not issued unrestricted.

Two tree properties make it safe rather than decorative, and both are recursive queries in the database rather than bookkeeping in the application: spend is summed over a key *and all its descendants* (otherwise minting a child is a trivial way around your own budget), and revocation *cascades over the subtree* (otherwise revoking a key does nothing to an attacker who delegated first). Depth is capped, and tier is inherited rather than settable — because tier selects routing, and on the confidential tier it selects which hardware attests the call.

The intended pattern is one short-lived, tightly-scoped key per task: a runaway sub-agent is then bounded by construction rather than by somebody noticing.

06 Payment: prepaid USDT and x402

Machines should be able to pay for compute without a card, a signup, or a human.

Accounts are funded by transferring **USDT** to a treasury address on a supported EVM chain and submitting the transaction hash. Verification is done by the gateway against its own JSON-RPC node, and it is deliberately strict: the receipt must show success, the log must be an ERC-20 `Transfer` emitted *by the configured token contract*, the recipient must be the treasury, the sender must be a wallet already linked to the account, and the transfer must have the required confirmations. Replay is prevented by a uniqueness constraint on the chain, transaction hash, and log index together, so a given transfer credits exactly once, globally.

Two of those checks exist because their absence is exploitable rather than merely untidy. Binding the sender to a linked wallet is what stops anyone from claiming a stranger's deposit by submitting its hash first. Requiring the log to come from the configured contract is what stops a lookalike token, deployed at an address a careless operator pasted, from crediting real balance. For the same reason token addresses are **operator-configured and deliberately not hardcoded**: a wrong address either fails to credit legitimate deposits or credits an unrelated token, and that decision should be made deliberately by someone checking the official contract list.

Deposits are likewise accepted in USDT on **Solana**, where a transfer is attributed to an account by a transaction memo rather than a linked sender wallet, and is verified only at finalized commitment. In production, deposits are live on Ethereum, Arbitrum One, and Solana. Metering and the ledger run on every deployment; whether an unpaid request is *refused* is a deployment flag, so billing can be observed against real traffic before it is enforced.

For agents that would rather not pre-fund, the gateway speaks **x402**. An out-of-credit request returns `402` with an `accepts` array describing exactly how to pay; the agent settles and retries with an `X-PAYMENT` header, which is verified and credited before the handler runs. The scheme is named `sable-usdt-onchain` rather than the canonical gasless `exact` flow, because USDT implements neither EIP-3009 nor EIP-2612 and a pull-based settlement is simply unavailable for it. Naming the scheme honestly is cheaper than explaining later why the standard one did not work.

ON TOKENS

The networks that paid for compute in their own token did not discover a new economic primitive; they discovered that paying for existence rather than for work attracts fabricated supply, and that a payout denominated in something the recipient cannot spend is not a payout. Compute is bought and sold here in dollars. If supply is ever paid, it is paid in dollars too. None of that has changed, and none of it will.

What has changed since the previous version of this document, which said Sable would not issue a token at all, is that in August 2026 the team launched **SABL**, a community token on Solana (mint `DaPayqzdCXcrmvgz9Wx7MySipXxcSofGPtkMgVdqpump` ; fixed supply, mint and freeze authority revoked). That version stated, as plainly as everything else in this paper, that SABL had no role in the platform and was not accepted as payment.

As of 2026-09-02 that stance is reversed on exactly one point. SABL gains a single utility: it can be used to pay for Sable compute and Sable Pro subscriptions, and paying in SABL burns it and earns a discount. This rail is **rolling out**, not yet live on the deployment. The guardrails around it are firm. Paying in SABL is optional and never required, because Sable is always priced in dollars and payable in USDT. It does not gate access: nothing on Sable requires holding SABL. It confers no yield, no revenue share, no governance, and no claim on Sable Network. It is a consumptive payment utility, not an investment. And supply-side operators, when they exist, are paid in dollars, never token emissions. The only official mint address is the one above, republished on Sable's official channels; anything else using the name is an impersonation. Distribution, as disclosed by the team and checkable on-chain: **8% of supply is held by the project treasury; the remaining 92% circulates on the open market.**

07 Verifiable receipts

A guarantee you can check beats a guarantee you have to believe.

Every billable response carries a **signed receipt**: a compact, metadata-only statement of what was processed and under what terms, signed with a **secp256k1** key over an **EIP-191** digest. The signing key is derived deterministically from the deployment's master key, and its address is published. Inference and compute use distinct shapes — `v:1` describes a token-metered call, `v:2` a unit-metered execution — so a verifier is never asked to reinterpret one as the other.

```
{
  "v": 2,
  "request_id": "3f9c...",
  "content_fingerprint": "a1b2c3d4...", // sha256 prefix, NOT content
  "kind": "sandbox", "resource": "python:3.12-slim",
  "unit": "vcpu_seconds", "quantity": 12,
  "cost_micro_usd": 178,
  "status": "ok", "exit_code": 0, "duration_ms": 6042,
  "logging": "metadata-only",
  "issued_at": "2026-08-07T12:00:00Z"
}
```

Because the scheme is standard secp256k1 / EIP-191, a receipt verifies with tooling the wider ecosystem already trusts — no Sable-specific library required:

```
// viem — verify against the published signer address
await verifyMessage({ address, message: atob(receipt), signature }) // → true
```

The gateway also exposes `GET /v1/receipts/pubkey` and `POST /v1/receipts/verify` for parties who prefer a hosted check. Non-streaming responses carry the receipt in headers; streaming responses emit it as a trailing `sable.receipt` event, since the headers are long gone by then.

It is worth being precise about what a receipt does and does not establish, because overstating it would undo the point. A gateway-signed receipt proves that *this deployment attests* to having processed a payload with that fingerprint, under that tier, at that metered cost — and that no field has been altered since. It is an accountable statement by a named party, not a proof of physics. What upgrades it into something stronger is the attestation block described next: the same envelope, with the enclave's own verified measurement inside it and the response bound to the key that measurement committed to. The shape does not change, so an integration written today inherits the stronger guarantee without a code change.

08 Confidential execution & attestation

Verify the hardware before you send it anything — and refuse rather than downgrade.

On the confidential tier, a request is routed only to a backend running inside an **Intel TDX** trusted execution environment, and only after the gateway has cryptographically verified that backend's attestation. Verification is real, not a label: the DCAP quote is verified in-process with a pure-Rust verifier against Intel collateral, the report's application measurement must match an operator-pinned **allowlist**, and the TCB status must be up to date. Anything less **fails closed** — the request is refused with an error, never served in plaintext by a different path.

Two properties are what make this more than attestation theatre.

Response binding

An attestation proves that a correctly-measured enclave exists. It does not, by itself, prove that *your* response came from it. The gateway therefore verifies a per-response signature against the signing key committed inside the quote's `report_data` at a fixed offset, and only then does the receipt's `response_bound` flag become `true`. Before that check it is explicitly `false`, and the receipt says so. Publishing the weaker state honestly is the difference between a verifiable claim and a marketing one.

Attest-then-call, enforced by types

Attesting one enclave and then sending the payload to another would produce a receipt describing hardware that never saw the request — the worst failure this system could have, and an easy one to introduce during a refactor. So attestation returns a *handle*, and the call and signature-verification functions take that handle rather than a name or an index. Routing to an enclave you did not attest is not a rule to remember; it does not compile.

The measurement pin is an allowlist rather than a single value for an operational reason learned the hard way: pinning one value turns every legitimate backend redeploy into a silent outage, which happened twice before the mechanism changed. An operator can now pre-authorize the next build. A background loop re-verifies attestation continuously, and a fail-closed window raises an operator alert within roughly two minutes rather than being discovered days later.

An honest statement of the current boundary: the enclave is operated by a confidential-computing provider, not by Sable. What is verified is that the workload runs inside genuine TDX hardware with a measurement we pinned, and that the response came from the key that hardware committed to. What is *not* yet true is that Sable owns the enclave — that is the direction, and it changes the trust story from "a verified third party" to "verified hardware, whoever runs it." The verification machinery is indifferent to which, which is the point.

09 Sandbox compute

Agents write code. Something has to run it, and the same rules should apply.

`POST /v1/sandboxes` executes a short-lived job — an image, an argv, code on stdin, a timeout, a vCPU and memory request — and returns its stdout, stderr, and exit code exactly once. It is billed in **vCPU-seconds and gigabyte-seconds**, on the same key and the same balance as inference. In production the work runs on a **Sable-operated house node** under gVisor isolation, enrolled through the same registry, heartbeat, and lease machinery a future fleet would use — one machine today, and the listing says so.

Pricing memory as well as CPU is not a rounding detail. Memory is what actually bounds how many sandboxes a host can run concurrently, so charging for CPU alone means the cheapest possible request can consume the capacity you had left to sell. Any metering scheme that ignores the binding resource is a scheme that rewards exhausting it.

The privacy contract applies without exception. The submitted code, its environment variables, and both output streams cross the module in-frame, are returned once, and are never persisted or logged — the table has no column that could hold them. This has a consequence worth stating rather than burying: a run replayed via its idempotency key returns the recorded receipt with a `replayed` marker and **empty output**. The metadata was kept; the output never was.

Defaults are chosen so that the safe posture is the one you get by not thinking about it: **network egress is denied unless requested**, the filesystem is read-only with scratch space in memory, capabilities are dropped, privilege escalation is disabled, and code is fed over stdin so it never appears in a process listing. A local Docker backend exists for development and single-tenant self-hosting and logs a loud warning at boot, because a container is not an isolation boundary for hostile code and pretending otherwise in production would be indefensible.

Failure classification is part of the billing contract. A non-zero exit is the caller's own program failing: it ran, and it is billed. A timeout held the slot and is billed. But a backend that was never reachable, or a runner that failed to start, is recorded and **costs nothing** — an early version billed for runs that never happened because a daemon-level failure exit code went down the success path, which is the kind of bug that is only ever found by running the thing for real.

10 What is built, and what is not

The most load-bearing section in this document.

Infrastructure whitepapers routinely describe a planned system in the present tense. We think that is the single most corrosive habit in this category, so the table below separates what the code does today from what it does not.

CAPABILITY	STATUS	NOTE
OpenAI-compatible chat & embeddings, streaming	Built	Multi-provider failover; 4xx short-circuits
Anthropic Messages API	Built	Translates and delegates to the chat path — not a second implementation
Sealed ingress, metadata-only persistence	Built	The privacy contract, §03
Signed receipts + public verification	Built	secp256k1 / EIP-191
Metered sandbox execution	Built	Live in production on a Sable-operated house node (gVisor; egress denied by default)
Credit ledger, pre-auth holds, spend caps	Built	Enforcement is a deployment flag; metering runs regardless
USDT deposits, x402 settlement	Built	Live on Ethereum, Arbitrum One, and Solana; sanctions screening is a configurable hook, off by default
Delegated sub-keys	Built	Monotone clamping, subtree spend, cascading revoke
Remote MCP server	Built	Bearer-key auth; OAuth 2.1 / DCR is <i>not</i> implemented
TDX attestation, fail-closed, response binding	Built	On a third-party enclave; availability depends on a verified backend
Double-blind private lane (pinned second intermediary)	Built	Inference only; pinned routing never fails over; not yet on the sandbox surface
Node registry, scheduler & node agent	Built	Enrollment, heartbeats, transactional per-job leases — serving exactly one Sable-operated house node
Sable Vault — private RWA registry	Built	Sealed-at-rest asset metadata and positions, per-asset hash-chained ledgers, signed selective-disclosure proofs, Solana memo-transaction anchoring. Registry infrastructure: entries are issuer-declared; no custody, no offers, no legal enforcement of ownership; attested + anchored, not zero-knowledge
Sable-owned enclaves	Not built	Direction — §11
Confidential <i>sandboxes</i> (host cannot read your code)	Not built	The next major build; needs a TDX host
Sealed env delivery; node-countersigned receipts (v:3)	Not built	Arrive with the operator programme
Third-party operators, earnings, payouts	Not built	Zero third-party machines serve traffic today

Multi-region routing	Not built	A region pin is honoured only if it matches this deployment's declared region, and refused otherwise
On-chain node registry, staking, slashing	Not built	Contingent on operators existing at all
Pay for Sable in SABL (optional): burns the token, earns a discount	Rolling out	Optional and never required. Sable is always priced in dollars and payable in USDT; does not gate access; no yield, revenue share, governance, or claim. \$06
Token-incentivized supply, token-gated access	Never	\$06
SABL community token (one optional pay-in utility, rolling out)	Exists, Aug 2026	\$06

Region pinning deserves its own sentence, because the previous version of this document described it incorrectly. A pinned request used to be mapped onto one of four labelled nodes that did not exist, and the caller's chosen region was then stamped onto a signed receipt — a jurisdiction claim backed by nothing. Today a pin is honoured only when it matches the region the deployment actually declares, and refused otherwise. Fewer requests succeed. The ones that do mean something.

11 Direction: compute for rent

The machines are the commodity. The trust layer over them is not.

The compute behind the API today is rented from providers, and the intent is to keep going in that direction rather than to buy hardware: first machines Sable rents and enrolls itself, then vetted third-party operators, and open supply last if ever. The first step is no longer hypothetical: the control machinery a fleet requires — a node registry with hardware descriptors and per-node credentials, heartbeats, and a scheduler that claims a per-job lease transactionally before dialing a node — is built and serving production sandbox traffic, on exactly one node, which Sable operates. Third-party supply remains gated on demand evidence and a compliance programme, not on code. What Sable would sell is never the machine — it is gateway-authoritative metering, hard budgets, signed receipts, fail-closed attestation, and agent-native payment *over hardware nobody has to trust*.

This ordering is a conclusion, not a preference. The record of the last several years is unambiguous about which parts work. Vetted supply with dollar rails works. Selling *outcomes* rather than raw machines works, and is the only model under which heterogeneous supply has ever held together. What fails, repeatedly and in the same way, is paying for existence rather than for work — which reliably produces fabricated supply — along with points and emissions treadmills, volunteer swarms, verification-first marketplaces that never found buyers, and undifferentiated machine listing, where the market is already won by incumbents.

The gap nobody fills is the one Sable already occupies: signed per-request receipts, fail-closed attestation, prepaid stablecoin and x402, and budget-bounded delegated credentials. That is a trust-and-payments layer, and it is what makes commodity compute sellable at a premium — the buyer is paying for the proof, not the silicon.

The sequencing follows from the same evidence. Demand comes before supply, because building the operator half first reproduces the failure mode of every exchange with no sellers. The wedge is *provable* compute rather than *cheap* compute, because at commodity prices nobody buys code execution on a stranger's readable machine — the buyer for whom this is better rather than merely cheaper is the one who needs their code kept from the host, or a portable proof of what ran.

A RULE WE HOLD OURSELVES TO

Sable will not describe itself as a marketplace, a network of operators, or decentralized infrastructure in the present tense until independent operators are actually serving a meaningful share of paid work. Until then this section is labelled as direction, the node listing reports only things whose existence can be checked, and any page inviting operators says outright that zero third-party machines serve Sable traffic. An exchange with no sellers is the most common artefact in this industry. We would rather ship late than describe one.

12 Threat model & trust boundaries

What is protected now, what is not, and which of those we intend to change.

Defended today

- **In-transit interception.** Payloads are sealed on ingress; plaintext never crosses the network between caller and dispatch.
- **At-rest retention.** No prompt, completion, submitted code, or output stream is ever written down. A subpoena, a breach, or a curious engineer finds metadata — never content.
- **Silent substitution and tampering.** A signed receipt lets a client detect any mismatch between what was claimed and what was delivered.
- **Unattested confidential execution.** A confidential request whose backend fails verification is refused, not downgraded — including when the failure is our own misconfiguration.
- **Budget exhaustion by a runaway agent.** Pre-auth holds, subtree-aware spend caps, and cascading revocation bound the blast radius of a sub-agent you no longer control.
- **Double execution and double billing.** Idempotency is enforced by database constraints rather than application memory, at both the billing and the execution layer.

Not yet solved — and how it changes

- **The gateway operator at the egress frame.** On the standard tier, plaintext is momentarily visible to the gateway process. Confidential inference removes this for the model call today; moving the shim itself into an enclave Sable owns removes it generally.
- **The sandbox host.** A runner host can read the code it executes and the environment handed to it. This is the honest gap, it is the reason confidential sandboxes are the next major build, and it is why a sandbox on a machine we do not control is not offered at all rather than offered quietly.
- **The upstream model host on the standard tier.** Decryption is required because a third-party provider necessarily sees its input. The confidential tier is the answer; the standard tier does not pretend otherwise, in the product or in the docs.
- **Availability.** A single gateway is a single point of failure. Several of its subsystems still hold per-process state, so horizontal scale is an engineering prerequisite rather than a configuration change.

We state the current boundary plainly because a confidentiality product that overstates its guarantees is worse than one that makes none: it manufactures confidence that is then acted on. The roadmap is, in effect, the systematic shrinking of this boundary — and every step of it should be checkable from outside, by a buyer who has no reason to take our word for anything.

13 Conclusion

The agent economy will route an enormous volume of machine intent through a small number of compute endpoints. Whether that layer becomes an unaccountable one — where nobody can say what ran, on what, for whom, or how much — is being decided now, by the defaults the infrastructure ships with.

Sable's bet is that the durable position is not the hardware and not the model, but the layer that makes a unit of compute *provable*: metered where the metering can be trusted, bounded before the money moves, attested where the hardware allows it, and signed into a receipt that outlives the request and does not require trusting the party that issued it. Build that, and the machine underneath stops needing to be trusted — which is exactly what makes it replaceable, and exactly why the trust layer is the part worth owning.

**We earn trust through architecture, not assurances. Promises don't scale.
Verification does.**

A Appendix – API surface

A representative slice. Full reference at buildsable.com/docs.

METHOD	PATH	PURPOSE
POST	/v1/chat/completions	OpenAI-shape chat; stream supported; accepts <code>sable_privacy_tier</code> , <code>sable_region</code> , <code>sable_scrub</code>
POST	/v1/embeddings	OpenAI-shape embeddings, same sealed path
POST	/v1/messages	Anthropic Messages API; translates and delegates to the chat handler
POST	/v1/sandboxes	Metered code execution; accepts <code>Idempotency-Key</code>
POST	/v1/mcp	Remote MCP server (JSON-RPC): chat, run code, verify receipt, attestation, balance, models
POST	/v1/keys/delegate	Mint a bounded child of the calling key — parent-key authenticated
GET	/v1/credit	Key-authenticated balance — an agent’s remaining runway
GET	/v1/models	Catalog under stable Sable IDs, with per-token pricing
GET	/v1/attestation	Live verified TEE attestation — pre-flight, before you send anything
GET	/v1/receipts/pubkey	Receipt-signer address & scheme
POST	/v1/receipts/verify	Verify a {receipt, signature} — public, no key required
GET	/v1/billing/methods	Treasury address, chains, token contracts, confirmations
GET	/v1/nodes	What is actually serving: this gateway and its configured backends

Response headers on every metered call: `x-sable-node`, `x-sable-region`, `x-sable-privacy-tier`, and the receipt triplet `x-sable-receipt` / `-sig` / `-signer`. Streaming responses emit the receipt as a trailing `sable.receipt` event instead, because the headers have already been flushed.